

EXTENDS *FiniteSets, Sequences, Naturals, TLC*

```
* Licensed to the Apache Software Foundation (ASF) under one
* or more contributor license agreements. See the NOTICE file
* distributed with this work for additional information
* regarding copyright ownership. The ASF licenses this file
* to you under the Apache License, Version 2.0 (the
* "License"); you may not use this file except in compliance
* with the License. You may obtain a copy of the License at
*
* http://www.apache.org/licenses/LICENSE-2.0
*
* Unless required by applicable law or agreed to in writing, software
* distributed under the License is distributed on an "AS IS" BASIS,
* WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
* See the License for the specific language governing permissions and
* limitations under the License.
```

This defines the YARN registry in terms of operations on sets of records.

Every registry entry is represented as a record containing both the path and the data.

It assumes that

1. operations on this set are immediate.
2. selection operations (such as $\forall$ and $\exists$ are atomic)
3. changes are immediately visible to all other users of the registry.
4. This clearly implies that changes are visible in the sequence in which they happen.

A Zookeeper-based registry does not meet all those assumptions

1. changes may take time to propagate across the *ZK* quorum, hence changes cannot be considered immediate from the perspective of other registry clients. (assumptions (1) and (3)).
2. Selection operations may not be atomic. (assumption (2)).

Operations will still happen in the order received by the elected *ZK* master

A stricter definition would try to state that all operations are eventually true excluding other changes happening during a sequence of action. This is left as an exercise for the reader.

The specification also omits all coverage of the permissions policy. This is something which can be tuned by the Resource Manager: who sets up

CONSTANTS

<i>PathChars</i> ,	the set of valid characters in a path
<i>Paths</i> ,	the set of all possible valid paths
<i>Data</i> ,	the set of all possible sequences of bytes
<i>Address</i> ,	the set of all possible address n-tuples
<i>Addresses</i> ,	the set of all possible address instances

RegistryEntry $\triangleq$ [
 The path to the entry
path : *Paths*,
 the data in the entry
data : *Data*
]

An endpoint in a service record
Endpoint $\triangleq$ [
API of the endpoint: some identifier
api : *STRING*,
 A list of address n-tuples
addresses : *Addresses*
]

A service record
ServiceRecord $\triangleq$ [
ID – used when applying the persistence policy
yarn_id : *STRING*,
 the persistence policy
yarn_persistence : *PersistPolicySet*,
 A description
description : *STRING*,
 A set of endpoints
external : *Endpoints*,
Endpoints intended for use internally
internal : *Endpoints*
]

Action Records

putAction $\triangleq$ [
type : “put”,
record : *ServiceRecord*
]

deleteAction $\triangleq$ [
type : “delete”,
path : *STRING*,
]

```

    recursive : BOOLEAN
]

purgeAction  $\triangleq$  [
    type : "purge",
    path : STRING,
    persistence : PersistPolicySet
]

mkNodeAction  $\triangleq$  [
    type : "mknode",
    path : STRING,
    parents : BOOLEAN
]

```

Path operations

parent is defined for non empty sequences

$parent(path) \triangleq SubSeq(path, 1, Len(path) - 1)$

$isParent(path, c) \triangleq path = parent(c)$

Registry Access Operations

Lookup all entries in a registry with a matching path

$lookup(Registry, path) \triangleq \forall entry \in Registry : entry.path = path$

A path exists in the registry iff there is an entry with that path

$exists(Registry, path) \triangleq lookup(Registry, path) \neq \{\}$

parent entry, or an empty set if there is none

$parentEntry(Registry, path) \triangleq lookup(Registry, parent(path))$

$isRootPath(path) \triangleq path = \langle \rangle$

the root entry

$isRootEntry(entry) \triangleq entry.path = \langle \rangle$

A path p is an ancestor of another path d if they are different, and the path d starts with path p

$isAncestorOf(path, d) \triangleq$
 $\wedge path \neq d$
 $\wedge \exists k : SubSeq(d, 0, k) = path$

$ancestorPathOf(path) \triangleq$
 $\forall a \in Paths : isAncestorOf(a, path)$

the set of all children of a path in the registry

$children(R, path) \triangleq \forall c \in R : isParent(path, c.path)$

a path has children if the *children()* function does not return the empty set
 $hasChildren(R, path) \triangleq children(R, path) \neq \{\}$

Descendant: a child of a path or a descendant of a child of a path

$descendants(R, path) \triangleq \forall e \in R : isAncestorOf(path, e.path)$

Ancestors: all entries in the registry whose path is an entry of the path argument
 $ancestors(R, path) \triangleq \forall e \in R : isAncestorOf(e.path, path)$

The set of entries that are a path and its descendants

$pathAndDescendants(R, path) \triangleq$
 $\vee \forall e \in R : isAncestorOf(path, e.path)$
 $\vee lookup(R, path)$

For validity, all entries must match the following criteria

$validRegistry(R) \triangleq$
 there can be at most one entry for a path.
 $\wedge \forall e \in R : Cardinality(lookup(R, e.path)) = 1$

There's at least one root entry
 $\wedge \exists e \in R : isRootEntry(e)$

an entry must be the root entry or have a parent entry
 $\wedge \forall e \in R : isRootEntry(e) \vee exists(R, parent(e.path))$

If the entry has data, it must be a service record
 $\wedge \forall e \in R : (e.data = \langle \rangle \vee e.data \in ServiceRecords)$

Registry Manipulation

An entry can be put into the registry *iff* – its parent is present or it is the root entry

$canPut(R, e) \triangleq$
 $isRootEntry(e) \vee exists(R, parent(e.path))$

put adds/replaces an entry if permitted

$put(R, e) \triangleq$
 $\wedge canPut(R, e)$

$$\wedge R' = (R \setminus \text{lookup}(R, e.\text{path})) \cup \{e\}$$

mknode() adds a new empty entry where there was none before, *iff*

- the parent *exists*
- it meets the requirement for being “put”

$$\begin{aligned} \text{mknodeSimple}(R, \text{path}) &\triangleq \\ \text{LET } \text{record} &\triangleq [\text{path} \mapsto \text{path}, \text{data} \mapsto \langle \rangle] \\ \text{IN } &\vee \text{exists}(R, \text{path}) \\ &\vee (\text{exists}(R, \text{parent}(\text{path})) \wedge \text{canPut}(R, \text{record}) \wedge (R' = R \cup \{\text{record}\})) \end{aligned}$$

For all parents, the *mknodeSimple* criteria must apply. This could be defined recursively, though as TLA+ does not support recursion, an alternative is required

Because this specification is declaring the final state of a operation, not the implemental, all that is needed is to describe those parents.

It declares that the *mkdirSimple* state applies to the path and all its parents in the set R'

$$\begin{aligned} \text{mknodeWithParents}(R, \text{path}) &\triangleq \\ &\wedge \forall p2 \in \text{ancestors}(R, \text{path}) : \text{mknodeSimple}(R, p2) \\ &\wedge \text{mknodeSimple}(R, \text{path}) \end{aligned}$$

$$\begin{aligned} \text{mknode}(R, \text{path}, \text{recursive}) &\triangleq \\ \text{IF } \text{recursive} &\text{ THEN } \text{mknodeWithParents}(R, \text{path}) \text{ ELSE } \text{mknodeSimple}(R, \text{path}) \end{aligned}$$

Deletion is set difference on any existing entries

$$\begin{aligned} \text{simpleDelete}(R, \text{path}) &\triangleq \\ &\wedge \neg \text{isRootPath}(\text{path}) \\ &\wedge \text{children}(R, \text{path}) = \{\} \\ &\wedge R' = R \setminus \text{lookup}(R, \text{path}) \end{aligned}$$

recursive delete: neither the path or its descendants exists in the new registry

$$\begin{aligned} \text{recursiveDelete}(R, \text{path}) &\triangleq \\ &\text{Root path: the new registry is the initial registry again} \\ &\wedge \text{isRootPath}(\text{path}) \Rightarrow R' = \{[\text{path} \mapsto \langle \rangle, \text{data} \mapsto \langle \rangle]\} \\ &\text{Any other entry: the new registry is a set with any existing} \\ &\text{entry for that path is removed, and the new entry added} \\ &\wedge \neg \text{isRootPath}(\text{path}) \Rightarrow R' = R \setminus (\text{lookup}(R, \text{path}) \cup \text{descendants}(R, \text{path})) \end{aligned}$$

Delete operation which chooses the recursiveness policy based on an argument

$$\begin{aligned} \text{delete}(R, \text{path}, \text{recursive}) &\triangleq \\ \text{IF } \text{recursive} &\text{ THEN } \text{recursiveDelete}(R, \text{path}) \text{ ELSE } \text{simpleDelete}(R, \text{path}) \end{aligned}$$

Purge ensures that all entries under a path with the matching *ID* and policy are not there afterwards

$$\begin{aligned} \text{purge}(R, \text{path}, \text{id}, \text{persistence}) &\triangleq \\ &\wedge (\text{persistence} \in \text{PersistPolicySet}) \\ &\wedge \forall p2 \in \text{pathAndDescendants}(R, \text{path}) : \\ &\quad (p2.\text{yarn_id} = \text{id} \wedge p2.\text{yarn_} = \text{persistence}) \Rightarrow \text{recursiveDelete}(R, p2.\text{path}) \end{aligned}$$

resolveRecord() resolves the record at a path or fails.

It relies on the fact that if the cardinality of a set is 1, then the **CHOOSE** operator is guaranteed to return the single entry of that set, iff the choice predicate holds.

Using a predicate of **TRUE**, it always succeeds, so this function selects the sole entry of the lookup operation.

$$\begin{aligned} \text{resolveRecord}(R, \text{path}) &\triangleq \\ \text{LET } l &\triangleq \text{lookup}(R, \text{path}) \text{ IN} \\ &\wedge \text{Cardinality}(l) = 1 \\ &\wedge \text{CHOOSE } e \in l : \text{TRUE} \end{aligned}$$

The specific action of putting an entry into a record includes validating the record

$$\begin{aligned} \text{validRecordToPut}(\text{path}, \text{record}) &\triangleq \\ &\text{The root entry must have permanent persistence} \\ &\text{isRootPath}(\text{path}) \Rightarrow (\text{record}.\text{yarn_persistence} = \text{"PERMANENT"} \vee \text{record}.\text{yarn_persistence} = \text{""}) \end{aligned}$$

putting a service record involves validating it then putting it in the registry marshalled as the data in the entry

$$\begin{aligned} \text{putRecord}(R, \text{path}, \text{record}) &\triangleq \\ &\wedge \text{validRecordToPut}(\text{path}, \text{record}) \\ &\wedge \text{put}(R, [\text{path} \mapsto \text{path}, \text{data} \mapsto \text{record}]) \end{aligned}$$

The action queue can only contain one of the sets of action types, and by giving each a unique name, those sets are guaranteed to be disjoint

$$\begin{aligned} \text{QueueInvariant} &\triangleq \\ &\wedge \forall a \in \text{actions} : \\ &\quad \vee (a \in \text{PutActions} \wedge a.\text{type} = \text{"put"}) \\ &\quad \vee (a \in \text{DeleteActions} \wedge a.\text{type} = \text{"delete"}) \\ &\quad \vee (a \in \text{PurgeActions} \wedge a.\text{type} = \text{"purge"}) \\ &\quad \vee (a \in \text{MknodeActions} \wedge a.\text{type} = \text{"mknode"}) \end{aligned}$$

Applying queued actions

$$\begin{aligned} \text{applyAction}(R, a) &\triangleq \\ &\vee (a \in \text{PutActions} \wedge \text{putRecord}(R, a.\text{path}, a.\text{record})) \\ &\vee (a \in \text{MknodeActions} \wedge \text{mknode}(R, a.\text{path}, a.\text{recursive})) \end{aligned}$$

$$\begin{aligned} & \vee (a \in DeleteActions \wedge delete(R, a.path, a.recursive)) \\ & \vee (a \in PurgeActions \wedge purge(R, a.path, a.id, a.persistence)) \end{aligned}$$

Apply the first action in a list and then update the actions

$$\begin{aligned} applyFirstAction(R, a) & \triangleq \\ & \wedge actions \neq \langle \rangle \\ & \wedge applyAction(R, Head(a)) \\ & \wedge actions' = Tail(a) \end{aligned}$$

$$Next \triangleq applyFirstAction(registry, actions)$$

All submitted actions must eventually be applied.

$$Liveness \triangleq \Diamond(actions = \langle \rangle)$$

The initial state of a registry has the root entry.

$$\begin{aligned} InitialRegistry & \triangleq registry = \{ \\ & [path \mapsto \langle \rangle, data \mapsto \langle \rangle] \\ & \} \end{aligned}$$

The valid state of the “registry” variable is defined as Via the *validRegistry* predicate

$$ValidRegistryState \triangleq validRegistry(registry)$$

The initial state of the system

$$\begin{aligned} InitialState & \triangleq \\ & \wedge InitialRegistry \\ & \wedge ValidRegistryState \\ & \wedge actions = \langle \rangle \end{aligned}$$

The registry has an initial state, the series of state changes driven by the actions, and the requirement that it does act on those actions.

$$\begin{aligned} RegistrySpec & \triangleq \\ & \wedge InitialState \\ & \wedge \Box[Next]_{vars} \\ & \wedge Liveness \end{aligned}$$

Theorem: For all operations from that initial state, the registry state is still valid

$$THEOREM \quad InitialState \Rightarrow \Box ValidRegistryState$$

Theorem: for all operations from that initial state, the type invariants hold

THEOREM $InitialState \Rightarrow \Box TypeInvariant$

Theorem: the queue invariants hold

THEOREM $InitialState \Rightarrow \Box QueueInvariant$